

这篇文章的标题为《动态本地可搜索对称加密》（Dynamic Local Searchable Symmetric Encryption），主要探讨了动态内存高效的可搜索对称加密（SSE）问题。这是首次针对动态内存高效SSE进行深入研究。文章的核心是提出了一种新的方法，称为“通用本地变换”（Generic Local Transform），该方法可以将具有特定特性的页面高效SSE方案转换为具有强本地性特性的SSE方案。

主要内容和贡献：

1. 动态内存高效SSE的定义：

- 动态内存高效的SSE意指同时实现动态页面高效SSE和动态本地SSE的目标[1][5]。

2. 通用本地变换：

- 文章提出了通用本地变换，展示了页面高效性和本地性之间的关系。该变换可以将页面高效的SSE方案作为基础，构建出本地性强的方案[6][12]。

3. 已有方案的局限性：

- 之前的SSE方案大多是静态的，限制了其应用。文章指出，现实中的许多应用需要动态性，这使得现有的本地和页面高效SSE的实用性受到严重限制[5]。

4. 技术细节：

- 文章详细讨论了如何在不泄露其他列表长度的情况下，将多个列表存储在不可信的服务器上，并介绍了多种分配机制来优化存储效率和页面效率[8][10]。

5. 结果和性能：

- 通过应用通用本地变换，文章展示了一个无条件的静态SSE构建，该构建在存储效率和读取效率上达到了 $O(1)$ 的形式。具体来说，与Cash和Tessaro在Eurocrypt 2014年提出的下界相比，该构建在效率上非常接近[2][4]。

摘要

首次针对动态内存高效的可搜索对称加密（SSE）问题进行了探讨。“内存高效”SSE 包含了局部 SSE 和页面高效 SSE 的目标。我们的方法引入了一个新的概念，称为 **通用局部变换**，该变换将具有特定特征的页面高效 SSE 方案转换为具有强局部性特性的 SSE 方案。我们基于这一联系提出了多个结果：

1. 对于页面高效的SSE，构建了一个具有页面效率 $O(\log \log N)$ 和存储效率 $O(1)$ 的动态方案，称为LayeredSSE。
2. 介绍通用局部变换，并将其与LayeredSSE相结合，在最长列表大小为 $O(N^{1-1/\log \log N})$ 的条件下，构建一个存储效率为 $O(1)$ 、局部性为 $O(1)$ 、读取效率为 $\tilde{O}(\log \log N)$ 的动态SSE方案。
3. 通过将Bossuat等人在Crypto 2021中提出的Tethys方案的一个变体应用泛型局部变换，我们构建了一个无条件的静态SSE，其存储效率为 $O(1)$ ，局部性为 $O(1)$ ，读取效率为 $O(\log^\epsilon N)$ ，其中 ϵ 是任意小的常数且 $\epsilon > 0$ 。

介绍

• 可搜索对称加密（SSE）

客户端将一组文档的存储外包给不受信任的服务器，希望通过向服务器发出搜索查询来保留搜索文档的能力。

在动态SSE的设置中，客户端还可以发出更新查询，以修改数据库的内容。服务器必须能够正确处理所有查询，同时尽可能少地学习有关客户端数据和查询的信息。作为加密数据计算的特例，可以用诸如全同态加密的通用解决方案来实现SSE。然而这会造成很大的性能损失，与SSE提供高性能解决方案的初衷相悖。因此，SSE以安全换取效率。例如，SSE方案通常会向服务器泄露查询的重复（**搜索模式**）和与查询匹配的文档的标识符（**访问模式**）。SSE的安全模型由泄露函数参数化，该函数指定了泄露到服务器的信息的性质。

• 局部性

为了实现单关键字搜索查询，服务器维护一个（加密的）**倒排索引**，其中每个关键字都映射到与关键字匹配的文档标识符列表。当客户端进行搜索时，只需要从服务器检索相应的列表，但服务器应该如何存储和访问这个列表呢？

一直天真的办法就是一个接一个地存储列表，然而给定列表在内存中的位置取决于其他列表的长度，这就泄露了有关这些列表的信息。解决这个问题的一种常见方法是将每个列表元素存储在内存中的随机位置，然而，对于几乎所有的现代存储介质来说，访问许多随机存储位置比访问一个连续区域要昂贵得多。为了定义这种花费，[CT14]引入了局部性的概念：简而言之，SSE方案的**局部性**是服务器为了回答查询而必须访问的不连续内存位置的数量。

上述两个极端的方法表明了安全和局部之间的冲突，事实上，这种冲突是固有的：如果安全的SSE方案具有**恒定的存储效率**（加密数据库的大小与明文数据库的大小成线性关系）和**恒定的读取效率**（服务器为回答搜索查询而读取的数据量与明文答案的大小呈线性关系），则它不可能具有恒定的局部性。

• 局部SSE构造

许多具有恒定局部性的SSE方案被提出，通常以超恒定读取效率为代价。在大伙的努力下，读取效率从 $\tilde{O}(\log N)$ 到 $O(\log^{2/3+\epsilon} N)$ 再到 $\tilde{O}(\log \log N)$ ，最后来到了 $\tilde{O}(\log \log \log N)$ 。当然了，这是在数据库中最长列表的大小有限时得到的结果。当需要这样的上限时，我们将称构造为**有条件的**。

局部性被引入作为内存访问的性能指标，假设在硬盘驱动器上实现。然而对于固态硬盘（闪存），性能主要取决于访问的内存页的数量。在这种情况下，正确的性能指标是页面效率。**页面效率**是指服务器为回答查询而读取的页面数量除以存储明文答案所需的页面数量。

目前为止，所有现有结构（包括本地和页面高效结构）的一个共同点是，它们纯然是静态的。然而，SSE的许多（如果不是大多数）应用都需要动态性。这种情况严重阻碍了本地和页面高效的SSE的适用性。

1.1 我们的贡献

同时针对动态页面高效SSE和动态局部SSE，引入一个名为“**通用局部变换**”的映射，该映射将具有某些特殊特征的页面高效SSE方案作为输入，并输出具有强局部属性的SSE方案。

- **动态页面高效SSE——LayeredSSE**：其核心是一种新的动态分配方案L2C。L2C是所谓的“2-choice”算法的加权变体。
- **通用局部变换**：在输入任何具有某些特殊功能的页面高效方案PE-SSE（称为页面长度隐藏SSE）时，通用局部变换会输出一个局部SSE方案Local[PE-SSE]。可以被视为将具有弱局部性特性的方案引导到具有更强局部性属性的方案中。
- **动态局部SSE**：通过将通用局部变换应用于LayeredSSE页面高效方案，我们立即得到动态SSE方案Local[LayeredSSE]
- **静态设置中的无条件局部SSE**：通过将泛型局部变换应用于Tethys的一个变体[BBF+21]，并结合[DPP18]启发的技术，我们得到了一个无条件的静态SSE方案，其中 $\varepsilon > 0$ 是一个任意小的常数，其存储效率为 $O(1)$ ，局部性为 $O(1)$ ，读取效率为 $O(\log^\varepsilon N)$ 。

这个方案不是前向安全的，有人说“对于动态方案，局部性和前向隐私是两个不可调和的概念”。局部性要求与相同关键字关联的标识符必须彼此靠近存储；而前向隐私要求插入新标识符的位置应独立于它所关联的关键字。

技术概述

- **One-choice allocation**

n 个球被扔进 n 个箱子，每个球都被插入到一个独立且均匀随机选择的容器中（通过散列球的标识符）。插入过程中，装载量最大的箱子包含 $O(\log N)$ 个球的概率很高

- **Two-choice allocation**

n 个球被扔进 n 个箱子，对于每个球，随机独立均匀地选择两个箱子，球被插入到两个箱子里包含最少球的一个。插入过程中，装载量最大的箱子包含 $O(\log \log N)$ 个球的概率很高

- **Cuckoo hashing**

这是一个经典的哈希方案，将 n 个球插入 $(2 + \varepsilon)n$ 个单元格中，其中每个单元格最

多只能放一个球。每个球随机独立地选择两个单元格进行插入。如果选中的单元格已被占用，则原有的球会被移动到另一个目标单元格，可能引发连锁反应。插入过程预期在 $O(\log n)$ 时间内完成。每个球最终存储在两个可能位置中的一个。负载最大的单元格的负载为 $O(1)$

2.1 Layered 2-Choice Allocation

在页面高效方案中，我们有一个包含 N 个项目的列表。还有一个页面大小 p ，表示物理内存页面中可以容纳的项目数量。服务器的内存被视为一系列页面。我们希望将列表存储在服务器内存中，同时考虑三个目标：

1. 为了存储所有列表，我们总共使用 $S \lceil N/p \rceil$ 页的服务器内存，其中 S 称为分配方案的存储效率。我们希望 S 尽可能小。
2. 任何长度为 l 的列表都可以通过访问服务器内存中最多 $P \lceil l/p \rceil$ 个页面来检索，其中 P 称为分配方案的页面效率。我们希望 P 尽可能小。
3. 最后，服务器为检索给定列表而访问的页面不应取决于其他列表的长度。

前两个目标正是装箱算法的目标。第三个目标是安全目标：它规定服务器执行的内存访问模式不应泄露某些信息。实现这三个目标的框架被形式化为 Data-Independent Packing (DIP)。

为了简化演示，我们将重点介绍所有列表的大小最多为一页的情况。

实例化 DIP 方案是使用布谷鸟哈希的加权变体。作者在这篇文章中希望构建一个动态 SSE，然而先前提到的 Tethys 算法并不适合，因为更新过程中会访问其他的单元格。相反，一个自然的想法是使用两种选择分配方案的加权变体，这种方法只需要将新项目插入到当前两个桶中包含较少项目的桶中。

为了实现这个想法，作者引入了 layered weighed 2-choice allocation algorithm, L2C。将球的可能权重划分为 $O(\log \log \lambda)$ 个子区间，并对每个子区间中的球独立执行决策过程。对于第一个子区间（保持最小权重），我们使用**加权单选**过程，而对于其他子区间，我们使用**未加权的二选过程**。

最自然的选择是将球存储在当前负载最小的箱子中，其中箱子的负载是它当前包含的球的权重之和。

LayeredSSE方案是通过在L2C之上添加一层加密和密钥管理而获得的

2.2 Generic Local Transform

- 构建本地SSE的两种主要范式：

1. 分配范式(allocation paradigm)：其通常使用多项选择分配方案或布谷鸟散列的变体。
2. 垫分法(pad-and-split approach)：根据列表的大小分别存储列表。最简单方法是将所有列表的长度填充到2的下一大幂。这产生了列表长度的 $\log N$ 个可能值。

-
- Overflowing SSE (OSSE)：在设置和更新过程中，它可能会拒绝存储一些列表元素。这些元素被称为Overflowing(溢出)。溢出元素则使用单独的机制进行存储。

通用局部变换的思想是使用OSSE存储数据库中除 $1/\log N$ 的一小部分之外的所有内容。然后，使用垫分法的变体来存储 $N/\log N$ 溢出元素。

- 安全问题：给定的列表可以完全存储在OSSE方案中，也可以仅部分存储，或者根本不存储。服务器应该无法区分这三种情况。

假设所有的列表都被补齐到2的下一大幂，对于垫分法的构建，我们创建 $\log N$ 个实例，我们将每个实例称为一个**层 (layer)**。

为了实现总体存储效率 $O(S)$ ，我们希望该层使用 $O(Sn)$ 存储来存储这些列表。为了实现读取效率 R ，该层还应该能够通过访问最多 Rl 个内存位置来检索给定的列表。这就是这一切结合在一起的地方：满足这些条件的SSE方案正是具有页面大小 l 、存储效率 S 和页面效率 R 的页面高效SSE方案。

用于每一层的页面高效方案还需要满足一些额外的属性：

1. 在搜索最多一页大小的列表时，列表的长度不应泄露。我们称此属性**页长度隐藏**。
2. 我们要求页面高效方案具有 $O(1)$ 个客户端存储。
3. 我们要求该方案在获取单个页面时具有局部性 $O(1)$ 。

2.3 ClipOSSE: an OSSE scheme with $O(N/\log N)$ Overflowing Items

“二维”一选择和二选择分配变体，目的是建立本地SSE。一选择变体的工作原理如下。考虑一个包含 N 个元素的SSE数据库。分配 $m = \tilde{O}(N/\log N)$ 个最初为空的桶。对于数据库中长度为 l 的每个列表，随机均匀地选择一个桶作为起点。随后，该列表的所有元素依次循环插入从该桶开始的连续桶中。因此，假设 $l \leq m$ ，所有列表元素都被放置

在 l 个连续的桶中，每个桶中有一个元素。与通常对一选择过程的分析非常相似的分析表明，在压倒性的概率下，负载最大的桶最多接收 $\tau = \tilde{O}(\log N)$ 元素。

而对ClipOSSE，我们从相同的前提开始，但“clip”桶的阈值为 $\tau = \tilde{O}(\log \log N)$ ，这导致导致最多 $O(N / \log N)$ 个元素以压倒性的概率溢出。

2.4 Dynamic Local SSE with $O(\log \log N)$ Overhead

通过使用以ClipOSSE为底层OSSE、LayeredSSE为页面高效方案的通用局部变换，我们得到了Local[LayeredSSE]。Local[LayeredSSE]方案的存储效率为 $O(1)$ ，局部性为 $O(1)$ ，读取效率为 $\tilde{O}(\log \log N)$ 。

2.5 Unconditional Static Local SSE with $O(\log^\epsilon N)$ Overhead

Tethys方案需要 $\omega(p \log \lambda)$ 的客户端存储空间用于保存客户端侧的缓存（stash），而通用局部变换需要客户端存储限制为 $O(1)$ 。

为了减少客户端存储，将Tethys的缓存移至服务器端，并将其存储在一种高效的ORAM（Oblivious RAM）中，我们自然地获得了一个页面高效的SSE方案OramEthys。

为此，改进了一个具有常数局部性的ORAM（来源于[DPP18]），实现读取效率 $O(n^\epsilon)$ 的改进版本，称为LocORAM。

通过将Tethys存储在服务器端的LocORAM中，我们自然会获得一个页面高效的SSE方案OramEthys。

预备知识

对称可搜索加密

安全性

Definition 3.1 (Correctness): 如果对于所有数据库 DB 和 $N \in \mathbb{N}$ ，密钥 $K \leftarrow \Sigma.KeyGen(1^\lambda)$ ， $EDB \leftarrow \Sigma.Setup(K, DB)$ 和搜索序列，添加或删除查询 S ，如果数据库的大小保持在最多 N ，则搜索协议将返回该序列所有查询的正确结果，则SSE方案 Σ 是正确的。

Definition 3.2 (Semantic Security): 设 Σ 为SSE方案, $\mathcal{L} = (\mathcal{L}_{Stp}, \mathcal{L}_{Srch}, \mathcal{L}_{Udt})$ 为泄漏函数。如果对于所有PPT对手 $\mathcal{A}$, 存在一个PPT模拟器Sim使得

$$|\Pr[\text{SSEReal}_{\Sigma, \mathcal{A}}^{adp}(\lambda) = 1] - \Pr[\text{SSEIdeal}_{\Sigma, sim, \mathcal{L}, \mathcal{A}}^{adp}(\lambda) = 1]| = \text{negl}(\lambda)$$

则方案 Σ 是 $\mathcal{L}$ -自适应安全的

同样, 如果对于所有PPT对手 $\mathcal{A}$, 存在一个PPT模拟器Sim使得

$$|\Pr[\text{SSEReal}_{\Sigma, \mathcal{A}}^{sel}(\lambda) = 1] - \Pr[\text{SSEIdeal}_{\Sigma, sim, \mathcal{L}, \mathcal{A}}^{sel}(\lambda) = 1]| = \text{negl}(\lambda)$$

则方案 Σ 是 $\mathcal{L}$ -选择性安全的

真实实验 (SSEReal) : 攻击者与真实的加密方案交互, 观察系统行为。

理想实验 (SSEIdeal) : 攻击者与模拟器交互, 模拟器根据泄露函数的输出构造系统行为。

自适应安全性 (Adaptive Security) : 攻击者可以根据实验的反馈动态选择查询 (即查询是动态的)。

选择性安全性 (Selective Security) : 攻击者在实验开始前必须一次性提交所有查询 (即查询是静态的)。

直观地说, 语义安全保证了客户端和服务端之间的交互不会向服务器泄露任何信息, 除了给定查询的泄漏。

关键字 w 的查询模式 $qp(w)$ 是关键字 w 的先前搜索或更新查询的索引。

历史 $\text{Hist}(w)$ 由在设置期间插入的**与关键字 w 匹配的标识符列表**和**关键字 w 的更新历史**组成, 即每个被删除和插入的标识符。我们可以从 $\text{Hist}(w)$ 中检索每个关键字的插入标识符的数量 l_i 和删除标识符的数量 d_i 。

效率措施

Definition 3.3 (Read Pattern): 将服务器端存储视为包含加密数据库EDB的内存位置数组。在处理搜索查询 $\text{Search}(K, w_i, st_i; \text{EDB}_i)$ 或更新查询

$\text{Update}(K, (w_i, L'_i), st_i, \text{EDB}_i)$ 时, 服务器访问存储器位置 $m_1, \dots, m_h$ 。我们称这些位置为读取模式, 并用 $\text{RdPat}(op_i, in_i)$ 表示。

服务器处理搜索或更新时的读取模式 $RdPat(op_i, in_i) = m_1, \dots, m_h$ 为服务器访问的存储器的位置

在第 i 个操作中，服务器访问的内存位置集。比如，执行一个搜索操作时，服务器需要访问多个加密的列表， $RdPat(op_i, in_i)$ 就是这些加密列表所在的内存位置的集合。

Definition 3.4 (Locality): 如果对于任何 λ 、DB、N、序列S和任何 i ， $RdPat(op_i, in_i)$ 最多由 L 个不相交区间组成，则SSE方案具有局部性 L 。

这里的区间指的是内存位置的连续区间。例如，如果操作访问了内存地址

m_1, m_2, m_3, m_4 ，这些地址如果在物理内存中不是连续的，那么它们会形成多个不相交的区间。??????

$RdPat(op_i, in_i) = m_1, \dots, m_h$ 由 L 个不相交区间组成。对于一个操作，局部性是它所需访问的区间数。

局部性是指查询过程中服务器需要访问的**不连续内存区间的数量**。

Definition 3.5 (Read Efficiency): 如果对于任何 λ 、DB、N、序列S和任何 i ， $|RdPat(op_i, in_i)| \leq R \cdot P$ ，则SSE方案的读取效率为 R ，其中 P 是存储明文与关键字 w_i 匹配的所有（添加和删除）文档索引所需的存储位置数量（通过连接索引）。

$|RdPat(op_i, in_i)| \leq R \cdot P$ P 是存储所有（添加和删除）在明文中匹配关键字 w_i 的文档索引所需的内存位置数， R 是SSE方案的读取效率。

查询结果的**加密数据量**与原始查询结果的比值。

Definition 3.6 (Storage Efficiency): 如果对于任何 λ 、DB、N、序列S和任何 i ， $|EDB_i| \leq E \cdot |DB_i|$ ，则SSE方案具有存储效率 E 。

$|EDB_i|$ 是加密后的数据库的大小， $|DB_i|$ 是原始数据库的大小

存储效率就是加密后的数据库的大小与原始数据库的大小的比值

Definition 3.7 (Page Pattern): 将服务器端存储视为包含加密数据库EDB的页面数组。

在处理搜索查询 $Search(K, w_i, st_i; EDB_i)$ 或更新查询

$Update(K, (w_i, L'_i), op_i, st_i; EDB_i)$ 时，读取模式 $RdPat(op_i, in_i)$ 诱导多个页面访问 $p_1, \dots, p_{h'}$ 。我们称这些页面为页面模式，用 $PgPat(op_i, in_i)$ 表示。

Definition 3.8(Page Cost): SSE方案的页面成本为 $aX+b$, 其中 a 、 b 是实数, X 是固定符号, 如果对于任何 λ 、DB、 N 、序列 S 和任何 i , $|\text{PgPat}(op_i, in_i)| \leq aX + b$, 其中 X 是存储与明文中的关键字 w_i 匹配的文档索引所需的页面数。

页面模式小于页面成本

Definition 3.9(Page Efficiency): 如果对于任何 λ 、DB、 N 、序列 S 和任何 i , $|\text{PgPat}(\tau, \text{EDB})| \leq P \cdot X$, 则SSE方案具有页面效率 P , 其中 X 是存储与明文中的关键字 w_i 匹配的文档索引所需的页面数。

实际上访问的页面和应该要访问的页面的比值

4 Layered Two-Choice Allocation

我们描述了我们的算法L2C, 该算法允许将 n 个加权球分配到 m 个容器中, 其中每个球 b_i 的权重 $w_i \in [0, 1]_{\mathbb{R}}$ 。

L2C概述

我们将可能的权重 $[0, 1]_{\mathbb{R}}$ 集拆分为 $\log \log m$ 个子区间

$$[0, 1/\log m]_{\mathbb{R}}, (1/\log m, 2/\log m]_{\mathbb{R}}, \dots, (2^{\log \log m - 1}/\log m, 1]_{\mathbb{R}}$$

第一层: 单选分配 (1-choice allocation)

- 第一层包含权重 $w_i \leq 1/\log m$ 的数据元素, 采用单选分配, 即每个元素随机选择一个桶。
- 经典单选分配的分析结果表明, 负载的上界为 $O(\log m)$ 。但由于权重较小 (每个数据元素最多为 $1/\log m$) , 负载按权重调整后: 负载上界

$$\text{负载上界} = O\left(\frac{\log m}{1/\log m}\right) = O(\log \log m)$$

其他层：两选分配 (2-choice allocation)

- 对于每个区间 $(2^{i-1}/\log m, 2^i/\log m]$ 的数据元素，采用两选分配：

每个数据元素随机选择两个桶，插入负载较低的桶。

- 两选分配的理论分析结果表明，最大桶负载上界为 $O(\log \log n)$ ，其中 n 是分配的总数据元素数量。
- 在每层中，数据元素的权重最大值为 $2^i/\log m$ ，因此负载按权重调整后：

$$\text{负载上界} = O\left(\frac{\log \log m}{2^i/\log m}\right) = O(\log \log m)$$

全局负载上界

通过分层，将总负载分解为各层负载的叠加。各层的负载独立，且每层的负载均为 $O(\log \log m)$ 。因此，总体负载的上界仍然是：

$$O(\log \log m)$$

处理更新（拓展到动态场景）

1. 更新操作的具体步骤

在 L2C 中，处理更新的具体步骤如下：

- **插入新数据：**
 - 当一个数据元素的权重更新时（例如，插入一个新的数据元素或现有元素的权重发生变化），我们首先会根据新的权重计算它应该属于哪个区间。
 - 然后，我们根据新的权重区间进行分配。如果新权重依然在原来的区间内，数据元素就继续使用原来的桶。如果跨层，则需要跨层更新。
- **更新权重时的处理：**
 - **简单更新：**如果更新后的数据元素依然属于相同的权重区间，那么只需要将数据元素从原来的桶中移除并放入同一层次的另外一个桶中。

- **跨层更新**：如果更新后的权重超出了当前层次的权重区间，则需要将数据元素从当前层次的桶中移除，并重新根据新的权重区间重新选择一个桶。这个过程中会涉及到重新分配数据元素的位置。

2. 残留球 (Residual Balls)

当数据元素的权重发生更新，并且发生跨层时，L2C 引入了 **残留球** (residual balls) 的概念。这个概念的关键在于，虽然数据元素的权重更新导致它从原来的桶中移除并插入到新的桶中，但 **原桶仍然需要保留该数据元素的旧位置**，直到完全完成更新。

具体来说：

- 在更新过程中，数据元素仍然会被视为一个残留的元素，保持其在旧桶中的状态，以确保在数据更新过程中不会出现数据丢失或不一致的情况。
- 这一处理方式的好处是，L2C 依然能保证在更新操作中，负载的均衡性和桶的负载上限不会受到过多影响。

L2C的负载分析

• 定理1:

令 $\delta(\lambda) = 1$ 或者 $\delta(\lambda) = \log \log \log \lambda$ 。令 $w_{max} = poly(\lambda)$ 并且 $m = w_{max} / (\delta(\lambda) \log \log w_{max})$ 。如果 $\delta(\lambda) = \log \log \log \lambda$ ，我们要求 $m = \Omega(\lambda^{\frac{1}{\log \log \lambda}})$ ；否则要求 $m = \Omega(\lambda)$ 。令 $\{(b_i, w_i)_{i=1}^n\}$ 为具有唯一标识符 b_i 和权重 $w_i \in [0, 1]$ 的球。进一步，令 $S = \{(b_i, w_i)_{i=n+1}^{s+n}\}$ 为操作 $op_i \in \{L2C.InsertBall, L2C.UpdateBall\}$ 的序列，其中插入操作的输入 $in_i = (b_i, w_i, B_{\alpha_{i,1}}, B_{\alpha_{i,2}})$ ，更新操作的输入 $in_i = (b_i, o_i, w_i, B_{\alpha_{i,1}}, B_{\alpha_{i,2}})$ 。这里， b_i 表示在 op_i 执行之前具有权重 w_i 和旧权重 $o_i \leq w_i$ 的球的标识符。此外，通过 $\alpha_{i,1}, \alpha_{i,2} \leftarrow H(b_i)$ 选择箱子。

执行 $(B_i)_{i=1}^m \leftarrow L2C.Setup(\{(b_i, w_i)_{i=1}^n\})$ 和所有 $i \in [n+1, s+i]$ 的操作 $op_i(in_i)$ 。我们要求 $\sum_{i=1}^{n+s} w_i - o_i \leq w_{max}$ ，即所有操作后的总权重至多为 w_{max} 。

然后，在整个过程中，如果 H 被建模为随机预言机，则 $B_1, \dots, B_m$ 中最负载的箱子的负载至多为 $O(\delta(\lambda) \log \log w_{max})$ ，除了可以忽略的概率。

5 动态页面高效SSE

在这一部分，我们介绍了基于L2C的SSE方案LayeredSSE

我们将与关键词 w_i 匹配的标识符列表 L_i 解释为具有一定权重的球，并使用L2C来管理 m 个箱中的球。设 N 是数据库的最大大小， p 是页面大小， H 是一个映射到 $\{1, \dots, m\}^2$ 的哈希函数，其中 $m = \lceil w_{max} / (\log \log \log \lambda \cdot \log \log w_{max}) \rceil$ ， $w_{max} = N/p$ 。现在假设 $|L_i| \leq p$ ，即每个关键词最多具有 p 个相关关键词。

- $L2C.Setup(\{(w_i, L_i)\}_{i=1}^W, w_{max})$ ：我们将配对 (w_i, L_i) 解释为具有标识符 w_i 和权重 $|L_i|/p \in [0, 1]$ 的球，其中 L_i 是与关键词 w_i 匹配的（最多 p 个）标识符的列表。 (w_i, L_i) 的桶选择由 $\alpha_1, \alpha_2 \leftarrow H(w_i)$ 给出。在给定这些球的情况下运行算法1中定义的设置。
- $L2C.InsertBall((w, L), B_{\alpha_1}, B_{\alpha_2})$ ：将球 (w, L) 插入到算法1中的箱 B_{α_1} 或 B_{α_2} 中。
- $L2C.Update((w, L), L', B_{\alpha_1}, B_{\alpha_2})$ ：将球 (w, L) 的重量更新为 $|L \cup L'|/p$ ，如算法1中所示，并将标识符 L' 添加到列表 L 中。现在一个箱中包含球 $(w, L \cup L')$ 。如果新的重量位于不同的子间隔中，则一个箱中包含我们认为不再与 w 匹配的残余球 (w, L) 。

Algorithm 1 Layered 2-Choice Allocation (L2C)**L2C.Setup**($\{(b_i, w_i)_{i=1}^n\}, w_{\max}$)

- 1: Receive n balls b_i with weight w_i , and maximal total weight $w_{\max}$
- 2: Initialize $m = \lceil w_{\max}/(\delta(\lambda) \log \log w_{\max}) \rceil$ empty bins $B_1, \dots, B_m$
- 3: **for all** $i \in \{1, \dots, n\}$ **do**
- 4: Set $\alpha_1, \alpha_2 \leftarrow H(b_i)$
- 5: InsertBall($b_i, w_i, B_{\alpha_1}, B_{\alpha_2}$)
- 6: **Return** $B_1, \dots, B_m$

L2C.InsertBall($b_{\text{new}}, w_{\text{new}}, B_{\alpha_1}, B_{\alpha_2}$)

- 1: Receive bins $B_{\alpha_1}, B_{\alpha_2}$ and ball b_{new} with weight w_{new}
- 2: Assert that α_1, α_2 are the choices given by $H(b_{\text{new}})$
- 3: Split the set of possible weights $[0, 1]_{\mathbb{R}}$ into $\log \log m$ sub-intervals

$$[0, 1/\log m]_{\mathbb{R}}, (1/\log m, 2/\log m]_{\mathbb{R}}, \dots, (2^{\log \log m - 1}/\log m, 1]_{\mathbb{R}}$$

- 4: Choose $k \in \mathbb{N}$ minimal such that $w_{\text{new}} \leq 2^k/\log m$
- 5: **if** $k = 1$ **then**
- 6: Set $\alpha \leftarrow \alpha_1$
- 7: **else**
- 8: Let B_{α} be the bin with the least number of balls of weight in $\left(\frac{2^{k-1}}{\log m}, \frac{2^k}{\log m}\right]_{\mathbb{R}}$ among B_{α_1} and B_{α_2}
- 9: Insert ball b_{new} into bin B_{α}

L2C.UpdateBall($b_{\text{old}}, w_{\text{old}}, w_{\text{new}}, B_{\alpha_1}, B_{\alpha_2}$)

- 1: Receive bins $B_{\alpha_1}, B_{\alpha_2}$ that contain ball b_{old} with weight w_{old} and the new weight $w_{\text{new}} \geq w_{\text{old}}$
- 2: Assert that α_1, α_2 are the choices given by $H(b_{\text{new}})$
- 3: **if** $w_{\text{old}}, w_{\text{new}} \in \left(\frac{2^{k-1}}{\log m}, \frac{2^k}{\log m}\right]_{\mathbb{R}}$ for some k **then**
- 4: Update the weight of b_i to w_{new} directly
- 5: **else**
- 6: Mark b_{old} as residual ball (it is still considered as a ball of weight w_{old})
- 7: InsertBall($b_{\text{old}}, w_{\text{new}}, B_{\alpha_1}, B_{\alpha_2}$)

LayeredSSE

- LayeredSSE.KeyGen(1^λ) $\rightarrow K_{\text{Enc}}$
- LayeredSSE.Setup(K, N, DB)
- LayeredSSE.Search($K, w_i; \text{EDB}$) $\rightarrow B_{\alpha_1}^{\text{enc}}, B_{\alpha_2}^{\text{enc}}$, 其中 $(\alpha_1, \alpha_2) \leftarrow H(w)$
- LayeredSSE.Update($K, (w, L'), \text{add}; \text{EDB}$)

Algorithm 2 LayeredSSE

LayeredSSE.KeyGen(1^λ)

- 1: Sample K_{Enc} for Enc with security parameter λ
- 2: **return** $K = K_{\text{Enc}}$

LayeredSSE.Setup(K, N, DB)

- 1: Set $B_1, \dots, B_m \leftarrow \text{L2C.Setup}(\{(w_i, \text{DB}(w_i))\}_{i=1}^W, N/p)$
- 2: Fill bins $B_1, \dots, B_m$ up to size $p \cdot c \log \log \log(\lambda) \log \log(N/p)$ with zeroes
- 3: Set $B_i^{\text{enc}} \leftarrow \text{Enc}_{K_{\text{Enc}}}(B_i)$ for $i \in [1, m]$
- 4: **return** $\text{EDB} = (B_1^{\text{enc}}, \dots, B_m^{\text{enc}})$

LayeredSSE.Search($K, w; \text{EDB}$)

Client:

- 1: **return** w

Server:

- 1: Set $\alpha_1, \alpha_2 \leftarrow H(w)$
- 2: **return** $B_{\alpha_1}^{\text{enc}}, B_{\alpha_2}^{\text{enc}}$

LayeredSSE.Update($K, (w, L'), \text{add}; \text{EDB}$)

Client:

- 1: **return** w

Server:

- 1: Set $\alpha_1, \alpha_2 \leftarrow H(w)$
- 2: **return** $B_{\alpha_1}^{\text{enc}}, B_{\alpha_2}^{\text{enc}}$

Client:

- 1: Set $B_{\alpha_i} \leftarrow \text{Dec}_{K_{\text{Enc}}}(B_{\alpha_i}^{\text{enc}})$ for $i \in \{1, 2\}$
- 2: Retrieve ball (w, L) from B_α for appropriate $\alpha \in \{\alpha_1, \alpha_2\}$
- 3: Run $\text{L2C.UpdateBall}((w, L), L', B_{\alpha_1}, B_{\alpha_2})$
- 4: Set $B_{\alpha_i}^{\text{new}} \leftarrow \text{Enc}_{K_{\text{Enc}}}(B_{\alpha_i})$ for $i \in \{1, 2\}$
- 5: **return** $B_{\alpha_1}^{\text{new}}, B_{\alpha_2}^{\text{new}}$

Server:

- 1: Replace $B_{\alpha_i}^{\text{enc}}$ with $B_{\alpha_i}^{\text{new}}$ for $i \in \{1, 2\}$
-

安全性

2. 安全性证明：选择性安全 (Selective Security)

LayeredSSE 被证明是选择性安全的。为了证明这一点，作者通过以下思路构造了一个模拟器 (Simulator, Sim)：

- 初始设置 (Setup)：

- i. 模拟器只接收到泄漏信息 N (关键词总数)，重新计算桶的数量 m 并初始化空的桶。
- ii. 模拟器输出一个加密后的数据库 EDB' ，其中每个桶的内容通过一个 CPA 安全的加密算法 Enc 进行加密。
- iii. 因为加密算法是 IND-CPA 安全的 (即不可区分的)，并且真实实验中桶不会

溢出（除非概率极小），模拟器生成的 EDB' 和真实实验生成的结果是无法区分的。

- **搜索查询 (Search Queries) :** 模拟器根据查询模式 (Query Pattern qp) 来确定关键词是否已经查询过。如果没有查询过，则生成一个随机的伪关键词。如果查询过，则返回之前生成的伪关键词。由于客户端预处理关键词时使用了伪随机函数 (PRF)，原始关键词和伪关键词无法区分。
- **更新查询 (Update Queries) :**
 - 对于第一步的操作，模拟器的行为与搜索查询一致。
 - 对于第二步，模拟器接收两个桶 B_{α_1} , B_{α_2} ，对它们重新加密后返回给攻击者。由于桶被加密，攻击者无法区分真实实验和模拟实验。

3. 自适应安全性 (Adaptive Security) 的挑战

自适应安全性更复杂，因为攻击者可以根据之前的查询结果调整其策略。在这种情况下：

- **问题:** Theorem 1 只适用于选择性查询，而在自适应查询中，攻击者可能通过恶意的更新操作导致桶溢出，从而区别模拟实验和真实实验。
- **解决方案:** 对更新操作 (Update) 进行修改，增加一个检查机制：
 - i. 如果某次更新操作导致桶溢出，客户端会撤销此次更新并返回重新加密的原始桶。
 - ii. 这种检查保证了即使在自适应查询下，桶溢出的概率仍然是可以忽略的。

扩展

处理长列表

1. 列表 L 和大小限制:

- 假设有一个包含多个标识符（可能大于 p ）的列表 L 。在此方案中，我们将 L 拆分成大小最多为 p 的子列表。这意味着每个子列表中的元素数不会超过 p 。

2. 哈希表 T_{full} 和 T_{len} :

- 对于完整的子列表（即大小恰好为 p 的子列表），我们可以将这些子列表存储在一个名为 T_{full} 的哈希表中，服务器端将保存这些已加密的子列表。

- 对于不完整的子列表（即大小小于 p 的子列表），则继续按照 LayeredSSE 方案的原有方法进行处理。
- T_{len} 是另一张表，记录了每个关键字 w （对应的标识符为 l ）的子列表数量。具体来说， $T_{len}[w] = \lceil l/p \rceil$ 表示关键字 w 对应的标识符数量为 l 时，所需的子列表数（取上整）。

3. 客户端的角色：

- 客户端在执行搜索时需要知道子列表的数量，以便正确地从服务器获取数据。对于更新查询，客户端也需要知道当前的子列表数，以便在需要时将另一个完整的子列表插入 T_{full} 。

Setup（初始化阶段）

- 客户端将文档列表 L_i 切分为多个子列表 $L_{i,1}, \dots, L_{i,x_i}$ ，每个子列表大小不超过页面容量 p 。
- 使用伪随机函数 PRF 生成掩码 m_i ，并用其加密表 T_{len} 的内容。
- T_{len} 存储子列表的数量 x_i ，而 T_{full} 则存储子列表 $L_{i,j}$ 的内容。
- 不完整的子列表通过 L2C 算法分配到桶中，并将其视为新小球。
- 初始化完成后，返回加密的 T_{len} 、 T_{full} 和桶。

Search（查询阶段）

- 客户端根据关键词 w 生成掩码 m ，服务器使用该掩码解密 $T_{len}[w]$ 获得子列表数量 x 。
- 服务器从 T_{full} 中检索前 $x - 1$ 个子列表，并从桶中检索与 w 匹配的不完整列表。
- 服务器将加密的子列表和桶返回给客户端，客户端解密后获得所有匹配的标识符。

Update（更新阶段）

- 客户端生成掩码 m 并发送到服务器，服务器解密 $T_{len}[w]$ 获取当前子列表数量 x 。
- 服务器准备与关键词 w 相关的桶 $B_{\alpha_1}, B_{\alpha_2}$ 和可能的新桶 $B_{\alpha_3}, B_{\alpha_4}$ 。
- 如果更新后的子列表大小超过 p ，客户端将其拆分为两个子列表（一个完整子列表和一个不完整子列表），并插入到新桶中。
- 如果没有超出 p ，客户端直接更新当前桶内容。

Leakage Profile（信息泄露描述）

- 查询和更新操作会泄露子列表数量 $x = \lceil l/p \rceil$ 和每次更新是否完成一个子列表。

- 更新操作还会泄露更新后的子列表总数 $|l + |L'|]$ 。
- T_{len} 和 T_{full} 被加密，泄露内容受限与定义的泄露函数 $L_{len-hid}$ 。
- 安全性分析可以扩展到包括这些加密表的改进方案。

处理删除

1. 为了处理删除操作，作者采用了 [Bos16] 中的通用解决方案，并使用了两个 LayeredSSE 实例：
 - i. Σ_{add} 用于添加操作（新增标识符）。
 - ii. Σ_{del} 用于删除操作（删除标识符）。
2. **添加操作**：客户端通过 $\Sigma_{add}.Update$ 向关键词 w 添加新的标识符列表 L' 。
3. **删除操作**：客户端通过 $\Sigma_{del}.Update$ 向关键词 w 删除标识符列表 L' 。
4. **搜索操作**：

客户端从 Σ_{add} 和 Σ_{del} 中分别获取标识符列表 L_{add} 和 L_{del} 。

客户端通过从 Σ_{add} 的结果中删除从 Σ_{del} 获取的删除列表 L_{del} ，最终得到实际的匹配标识符列表 L ，即 $L \leftarrow L_{add} \setminus L_{del}$ 。

优化的RTT

1. **查询与更新的RTT**：
 - **搜索查询**：LayeredSSE 的搜索查询仅需要 1 次 RTT（往返时间）。
 - **更新查询**：更新查询需要 2 次 RTT，这意味着客户端需要等待两轮通信才能完成更新。
2. **减少更新RTT的优化方案**：
 - **Piggybacking（搭便车技术）**：通过搭便车技术，客户端将第二轮更新查询的响应暂存，并等待下一个查询（无论是更新还是搜索）。
 - **具体过程**：
 - a. 客户端首先处理更新查询的第一个步骤，将响应暂存。
 - b. 等待下一个查询（可能是搜索或其他更新）。
 - c. 客户端在下一个查询中将暂存的响应与新查询一起发送给服务器。
 - d. 服务器完成更新操作（存储接收到的桶并更新表格）并随后响应客户端的新查询。
3. **优化效果**：

通过搭便车技术，更新查询的RTT被减少到 1 次，与搜索查询相同，从而提高了系统的整体效率。

效率

定理 2 (LayeredSSE) :

- 假设：
 - N 为数据库的大小上界， p 为页面大小，且 $p \leq n^{1-1/\log \log \lambda}$ 。
- 正确性与安全性：
 - **正确性**：LayeredSSE 符合设计目标，能正确执行搜索和更新操作。
 - **语义安全性**：在加密方案 Enc 满足 IND-CPA 安全性，且哈希函数 H 被建模为随机 oracle 时，LayeredSSE 是 $L_{len-hid}$ ——适应语义安全的。
- 存储与效率：
 - 存储效率为常数 $O(1)$ 。
 - 页面效率为 $\tilde{O}(\log \log N/p)$ 。
 - 如果仅插入最大大小为 p 的列表，LayeredSSE 具有常数局部性 $O(1)$ 。

6 通用局部变换

6.1 预备知识

适用于Generic Local Transform的页面高效SSE (Page-efficient SSE)

- 接口扩展：

在Generic Local Transform中使用的页面高效SSE方案，需要扩展标准的SSE接口。具体扩展如下：

 - i. Setup阶段：
 - 新增了页面大小 p 参数。在设定数据库时，除了输入数据库的大小 N 外，还需要指定页面大小 p 。

- 该变换将基于不同的页面大小，创建多个底层页面高效SSE实例。

ii. Update阶段：

- 第二个参数 SSS 是一个文档标识符集合。在更新过程中，所有标识符应添加到关键词 w 对应的文档列表中。
- 标准的Update定义是将一个单一标识符添加到列表中，而在该方案中，允许 S 为空集合（即不添加任何内容）。

• 适用的页面高效SSE方案定义：

1. 客户端存储 $O(1)$ ：

- 客户端的存储开销为常数。

2. 局部性 $O(1)$ ：

- 在搜索和更新操作时，访问的文档列表（长度最多为一个页面）应具有常数局部性。

3. 页面长度隐藏（page-length-hiding）：

- 方案的泄露特性应确保页面长度被隐藏，即无法通过访问模式推测列表的长度。

溢出SSE

1. 概念介绍：

Overflowing SSE (OSSE) 是与标准SSE具有相同接口和功能的加密方案，唯一的区别是，在 Setup 或 Update 操作过程中，OSSE 可能会拒绝存储某些文档标识符。这些被拒绝的标识符称为 overflowing（溢出标识符）。

2. 溢出标识符的处理：

- 在 Setup 和 Update 操作的输出中，客户端会返回溢出标识符的集合。

3. 正确性定义的放宽：

相比标准SSE，OSSE的正确性定义被放宽。在 Search 操作中，只有那些没有溢出的匹配标识符需要被检索。换句话说，溢出的标识符不必出现在搜索结果中。

4. 使用场景：

OSSE 旨在作为一个组件，嵌入到一个更大的SSE系统中，专门用于存储溢出的标识符。其他标识符可以通过标准的SSE方案存储，而溢出的标识符则通过其他机制进行处理。

5. 与Generic Local Transform的关系：

引入OSSE概念是为了方便将 Generic Local Transform 的呈现分为两个部分：

- i. 使用OSSE方案存储大部分数据库。
- ii. 使用一组页面高效的SSE方案存储那些溢出的标识符。

6.2 动态二维单选分配

1. ClipOSSE 及其分解

- ClipOSSE 是 Generic Local Transform 的第一个组件，主要分为两个部分：
 - a. **分配方案 (1C-Alloc)**：定义了项目应该存储的位置，但本身不负责存储数据。
 - b. **SSE方案**：在1C-Alloc之上增加了加密、密钥管理和其他机制，将分配方案转化为完整的SSE。

超级桶 (Superbucket)

- **超级桶**：
 - 为了简化分配，1C-Alloc 将桶按大小划分为多个“超级桶”。
 - 当 $l = 2^i \leq m$ 时，**l-superbucket** 是由 l 个连续的桶组成的集合，索引为 $k \cdot l, k \cdot l + 1, \dots, (k + 1) \cdot l - 1$ ，其中 $k \leq m/l$ 。
 - 对于 $l > 1$ ，每个 l -超级桶包含两个 $l/2$ -超级桶。

1C-Alloc 的工作流程

- i. **Fetch**：
 - 给定一个关键词 w 和长度 l ，计算最小的 $l' = 2^{\lceil \log l \rceil}$ ，返回包含 $H(w)$ 的唯一 l' -超级桶。
- ii. **Add**：
 - **Add** 确保第一个 l 个元素插入的位置位于 $H(w)$ 所在的 l' -超级桶内。
 - **Add** 通过选择一个桶并将其分配给下一个待插入的元素，保证元素分配的正确性。

操作流程：

- **第一个元素**：返回 $H(w)$ 对应的桶。
- **第二个元素**：返回包含 $H(w)$ 的 2-超级桶中的另一个桶。
- **一般情况**：如果 S 是包含至少 $l+1$ 个桶的最小超级桶，**Add** 返回其中最左边未被使用的桶。

Algorithm 3 Dynamic Two-Dimensional One-Choice Allocation (1C-Alloc)	
1C-Alloc.Fetch(m, w, ℓ)	1C-Alloc.Add(m, w, ℓ)
1: $\ell' \leftarrow 2^{\lceil \log \ell \rceil}$	1: $\ell \leftarrow \ell \bmod m$
2: if $\ell' \geq m$ then	2: $\ell' \leftarrow 2^{\lceil \log(\ell+1) \rceil}$
3: return $\{0, \dots, m-1\}$	3: $i \leftarrow \lfloor H(w)/\ell' \rfloor$
4: else	4: if $\lfloor 2H(w)/\ell' \rfloor \bmod 2 = 0$ then
5: $i \leftarrow \lfloor H(w)/\ell' \rfloor$	5: return $\ell' \cdot i + \ell$
6: return $\{\ell' \cdot i, \dots, \ell' \cdot i + \ell' - 1\}$	6: else
	7: return $\ell' \cdot i + \ell - \ell'/2$

扩展

- 当列表大小 l 超过桶数 m 时，**Fetch** 返回所有桶，而 **Add** 则选取与 $l \bmod m$ 相同的桶。

剪切单选 OSSE

1. ClipOSSE 基本概念

- **桶的数量**：ClipOSSE 使用的桶数量 m 是 $O(N / \log \log N)$ ，其中 N 是文档数量。
 - 这个数量与文档的总数 N 成反比，但会考虑到一些对数因子，从而保证桶的数量适中，避免过多的桶造成存储浪费。
- **每个桶的大小**：每个桶可以容纳最多 $\tau = \lceil \alpha \log \log N \rceil$ 个文档标识符，其中 α 是常数。
 - 这意味着每个桶的容量根据 N 的规模调整，并且桶的容量是对数级别的。
- **填充与加密**：
 - 每个桶在存储之前都被 **填充**（填充到容量 τ ），即使有些位置为空。
 - 每个桶在存储之前都进行 **加密**，确保即使在服务器上，数据也是保密的。
 - 服务器只能看到加密后的桶，并且无法获取具体内容，因此每个桶对服务器来说是完全不透明的。

- **列表长度表 T:**

- 一个名为 T 的表存储了与每个关键词 w 匹配的文档列表的长度。这个表也被加密存储在服务器上。

2. ClipOSSE 的操作

Setup 操作概述:

- Setup 操作的目标是初始化数据库。这相当于从一个空数据库开始，逐个向其中插入文档标识符和关键词对。
- ClipOSSE 的 Setup 本质上是通过反复调用 Update 操作，将每个关键词及其对应的文档标识符插入到数据库中。

Search 操作:

- **目标:** 检索与关键词 w 匹配的所有文档标识符。

操作流程:

i. 调用 1C-Alloc:

ClipOSSE 使用 1C-Alloc(m, w, l) 函数来获取存储与关键词 w 匹配的文档标识符的桶索引。

- 这里，m 是桶的数量，w 是要查询的关键词，l 是当前已经存储的元素数量（即当前匹配的文档数量）。
- 该函数会返回一个桶的索引集合，这些桶存储了匹配关键词 w 的文档标识符。

ii. 从服务器获取桶:

客户端向服务器请求这些桶，并检索相应的加密桶数据。

iii. 解密桶:

客户端解密返回的桶数据，从中获取所有匹配的文档标识符。

总结:

- Search 操作的核心是通过 1C-Alloc 计算出存储匹配文档的桶索引，然后从服务器获取并解密这些桶，得到匹配的文档标识符。

Update 操作:

- **目标:** 向已存在的关键词列表中添加一个新的文档标识符。

操作流程：

i. 调用 1C-Alloc：

ClipOSSE 使用 1C-Alloc(m, w, l) 函数来决定新文档标识符应该插入到哪个桶中。

- m 是桶的数量， w 是关键词， l 是当前存储的元素数量。
- 该函数会返回一个桶索引，这个桶是下一个存储新元素的位置。

ii. 从服务器获取桶：

客户端向服务器请求该桶，并将其解密，得到桶中已有的文档标识符。

iii. 插入新元素：

客户端将新文档标识符添加到该桶中，并将更新后的桶重新加密。

iv. 检查是否溢出：

- 如果该桶已满（即超过了 τ 的容量），新元素会导致 **溢出** (Overflow)，需要通过 **Overflowing SSE** 机制处理。
- 溢出的元素将存储在一个单独的机制中，或者通过另一个桶来处理，确保溢出的标识符不会丢失。

v. 重新加密并返回桶：

客户端将更新后的桶加密后返回服务器，服务器将其存储。

总结：

- **Update** 操作通过 1C-Alloc 来计算插入位置，将新文档标识符插入到对应的桶中。如果桶已满，元素溢出并需要额外处理。

Algorithm 4 Clipped One-Choice OSSE (ClipOSSE)Global parameters: constants $d, \alpha \in \mathbb{N}^*$ **ClipOSSE.KeyGen**

- 1: Generate key K, K_{PRF} for Enc, PRF
- 2: **return** K, K_{PRF}

ClipOSSE.Setup(N, DB)

- 1: $m \leftarrow 2^{\lceil \log(N / \log \log N) \rceil}$
- 2: $\tau \leftarrow \lceil \alpha \log \log N \rceil$
- 3: $B_0, \dots, B_{m-1}, T, \text{EDB}, \text{clip} \leftarrow \emptyset$
- 4: **for all** each $(w, \{e_1, \dots, e_\ell\})$ in DB **do**
- 5: $K_w \leftarrow \text{PRF}_{K_{\text{PRF}}}(w)$
- 6: $T[w] \leftarrow \text{Enc}_{K_w}(\ell)$
- 7: **for all** t from 1 to ℓ **do**
- 8: $C \leftarrow \emptyset$
- 9: $i \leftarrow \text{1C-Alloc.Add}(m, w, t - 1)$
- 10: **if then** $|B[i]| < \tau$
- 11: $B[i] \leftarrow B[i] \cup \{e_i\}$
- 12: **else**
- 13: $C \leftarrow C \cup \{e_i\}$
- 14: **if** $|S| > 0$ **then**
- 15: $\text{clip} \leftarrow \text{clip} \cup (w, \ell, C)$
- 16: Let $B^{\text{Enc}}[i] = \text{Enc}_K(B_i)$ for each i
- 17: **return** $\text{EDB} = (T, (B^{\text{Enc}}[i])), \text{clip}$

ClipOSSE.Search(w)*Client (Search token):*

- 1: **send** $w, K_w = \text{PRF}_{K_{\text{PRF}}}(w)$

Server:

- 2: $\ell \leftarrow \text{Dec}_{K_w}(T[w])$
- 3: $S \leftarrow \text{1C-Alloc.Fetch}(m, w, \ell)$
- 4: **return** $\{B^{\text{Enc}}[i] : i \in S\}$

ClipOSSE.Update(w, e)*Client (Update token):*

- 1: **send** $w, K_w = \text{PRF}_{K_{\text{PRF}}}(w)$

Server:

- 2: $\ell \leftarrow \text{Dec}_{K_w}(T[w])$
- 3: $i \leftarrow \text{1C-Alloc.Add}(m, w, \ell)$
- 4: **send** $B^{\text{Enc}}[i]$
- 5: *Client:*
- 5: $B \leftarrow \text{Dec}_K(B^{\text{Enc}}[i])$
- 6: **if** $|B| < \tau$ **then**
- 7: $\text{clip} \leftarrow \emptyset$
- 8: $B \leftarrow B \cup \{e\}$
- 9: **else**
- 10: $\text{clip} \leftarrow \{e\}$

- 11: **send** $B' = \text{Enc}_K(B)$
- 12: *Server:*
- 12: $B^{\text{Enc}}[i] \leftarrow B'$
- 13: *Client:*
- 13: **return** clip

3. ClipOSSE 的关键特性

- **加密存储**：所有存储的桶都经过加密，确保数据隐私。即使服务器存储了所有数据，服务器也无法看到明文内容。
- **溢出机制**：当某个桶存储的文档数量超过 τ 时，新元素会导致 **溢出**，并使用 Overflowing SSE (OSSE) 机制来处理溢出的标识符。
- **桶索引与超级桶**：桶被分配到超级桶中，桶的数量和分配方式确保了数据存储的高效性和均匀性。
- **查询和更新操作的高效性**：通过 1C-Alloc, ClipOSSE 能够快速决定关键词对应的文档列表存储在哪些桶中，并进行高效的查询和更新。

通用局部变换

通用本地变换的目标是将一个 **长度隐藏的页面高效SSE (PE-SSE)** 方案转换为一个本地 SSE 方案 (Local[PE-SSE])，在保持高效性的同时，提供更好的局部性。这个过程结合了 ClipOSSE 和多个 PE-SSE 实例，来高效地存储和管理元素，同时保证数据隐私性。

输入和输出

- **输入**：一个 **长度隐藏的页面高效SSE (PE-SSE)** 方案。
- **输出**：一个本地SSE方案 (Local[PE-SSE])，具有更好的局部性和页面效率。

Local[PE-SSE] 中的结构

1. ClipOSSE:

- 用来存储大部分数据库中的元素。
- 元素会存储在这里，除非它们发生溢出。

2. PE-SSE 实例:

- 存储 ClipOSSE 溢出的元素。
- 有 n_{level} 个 PE-SSE 实例，记作 $PE-SSE_i$ ，其中 i 决定了页面大小为 2^i 。

3. 长度表 (T) :

- 存储每个关键词 w 的匹配列表长度，所有数据都是加密的。

元素存储

对于一个匹配 l 个元素的关键词 w :

1. 计算 $l' = 2^{\lceil \log l \rceil}$ ，即大于或等于 l 的最小 2 的幂次方。
2. 计算 $i = \log l'$ ，表示该元素的溢出存储实例是 $PE-SSE_i$ 。
3. 匹配的元素分布在两个地方：
 - ClipOSSE：存储未发生溢出的元素。
 - $PE-SSE_i$ ：存储溢出的元素。
4. **每个元素只会出现在一个位置**（即要么在 ClipOSSE 中，要么在 $PE-SSE_i$ 中）。

操作

1. 查询 (Search)

- **查询**操作通过查询两个结构来检索所有匹配的元素：
 - i. ClipOSSE：获取未溢出的元素。

- ii. PE-SSE_i : 获取溢出的元素 (如果有) 。
- 客户端将两个结构的输出合并, 获取所有匹配的元素。

2. 更新 (Update)

- **更新**操作处理向关键词 w 添加新元素 e :
 - i. 将更新请求转发到 ClipOSSE, 并得到输出:
 - 如果元素未溢出, 返回 $C = \emptyset$ (没有更新) 。
 - 如果元素溢出, 返回 $C = \{e\}$ (溢出的元素) 。
 - ii. 如果 $\lceil \log l \rceil = \lceil \log(l+1) \rceil$, 更新 PE-SSE_i 实例。
 - iii. 如果 $\lceil \log l \rceil < \lceil \log(l+1) \rceil$, 则更新为 PE-SSE_{i+1} , 并将溢出元素迁移到新的实例中。

Algorithm 5 Generic Local Transform (Local[PE-SSE])

Global parameters: constant $d \in \mathbb{N}^*$

Local[PE-SSE].KeyGen

- 1: Generate key K_{PRF} for PRF
- 2: **return** K, K_{PRF}

Local[PE-SSE].Update(w, e)

Client (Update token):

- 1: **send** $w, e, K_w = \text{PRF}_{K_{\text{PRF}}}(w)$

Server:

- 2: $C \leftarrow \text{ClipOSSE.Update}(w, e)$
- 3: $\ell \leftarrow \text{Dec}_{K_w}(T[w])$
- 4: $T[w] \leftarrow \text{Enc}_{K_w}(\ell + 1)$
- 5: **send** ℓ

Client:

- 6: **if** $\lceil \log \ell \rceil = \lceil \log(\ell + 1) \rceil$ **then**
- 7: $\text{PE-SSE}_{\lceil \log \ell \rceil}.\text{Update}(w, C)$
- 8: **else**
- 9: $i \leftarrow \lceil \log \ell \rceil$
- 10: $S \leftarrow \text{set of matches in } \text{PE-SSE}_i.\text{Search}(w)$
- 11: $\text{PE-SSE}_{i+1}.\text{Update}(w, S \cup C)$

Local[PE-SSE].Setup(N, DB)

- 1: $n_{\text{level}} \leftarrow \lceil N / \log^d N \rceil$
- 2: **for all** $(w, S) \in \text{DB}$ **do**
- 3: $K_w \leftarrow \text{PRF}_{K_{\text{PRF}}}(w)$
- 4: $T[w] \leftarrow \text{Enc}_{K_w}(|S|)$
- 5: $\text{EDB}, \text{clip} \leftarrow \text{ClipOSSE.Setup}(\text{DB})$
- 6: **for all** i from 0 to n_{level} **do**
- 7: $\text{DB}_i \leftarrow \{(w, C) : (w, \ell, C) \in \text{clip} \text{ and } 2^{i-1} < \ell \leq 2^i\}$
- 8: $\text{PE-SSE}_i \leftarrow \text{PE-SSE.Setup}(\lceil N / \log N \rceil, 2^i, \text{DB}_i)$

Local[PE-SSE].Search(w)

Client (Search token):

- 1: **send** $w, K_w = \text{PRF}_{K_{\text{PRF}}}(w)$

Server:

- 2: $i \leftarrow \lceil \log(\text{Dec}_{K_w}(T[w])) \rceil$
- 3: **return** $\text{ClipOSSE.Search}(w) \cup \text{PE-SSE}_i.\text{Search}(w)$

主要特性

1. 长度隐藏:

- 通过 ClipOSSE 和 PE-SSE 的组合, 数据的长度信息是隐藏的, 加密存储确保不会泄露列表的长度。

2. 隐私保护:

- 更新内容 C 的内容（包括是否为空）是加密的，无法被服务器解读。

3. 溢出处理：

- $PE-SSE_i$ 用来处理 ClipOSSE 溢出的元素，确保系统能有效地存储和检索大规模数据。

示例工作流程

假设关键词 w 匹配 7 个元素：

1. 计算 $l' = 2^{\lceil \log 7 \rceil} = 8$ ，所以 $i = 3$ 。
2. 未溢出的元素存储在 ClipOSSE 中。
3. 溢出的元素存储在 $PE-SSE_3$ （页面大小为 8）中。
4. 当新元素被添加时 ($l + 1 = 8$)：
 - 如果 $l + 1 \leq 8$ ，则更新 $PE-SSE_3$ 。
 - 如果 $l + 1 > 8$ ，则将溢出的元素迁移到 $PE-SSE_4$ （页面大小为 16）。

ClipOSSE 溢出

ClipOSSE 和通用本地变换 (Generic Local Transform)

1. ClipOSSE 中溢出项的数量分析

定理 3： 假设 ClipOSSE 接收的输入数据库大小为 (N) ，并且数据库中最长列表的大小为 $O(N/\text{polylog}(N))$ 。那么，对于任何常数 d ，存在一组参数选择，使得 ClipOSSE 中的溢出项数量为 $O(N/\log^d N)$ 。

解释：

- **溢出项的控制：** 该定理表明，随着数据库大小 N 增长，溢出项的数量增长速度相对较慢。通过合理的参数设置，ClipOSSE 可以有效管理溢出项数量。

2. 通用本地变换 (Generic Local Transform)

定理 4 (通用本地变换)：

假设 N 是数据库大小的上界，且 $PE-SSE$ 是一个合适的页面高效方案，具有页面效率 P

和存储效率 S ，则通过通用本地变换得到的 Local[PE-SSE] 方案是一个正确且安全的 SSE 方案，具有以下特性：

- **存储效率**： $O(S)$
- **局部性**： $O(1)$
- **读取效率**： $P + \tilde{O}(\log \log N)$

解释：

- **存储效率**：变换后的方案存储开销与 PE-SSE 相似，约为 $O(S)$ ，因此不会引入显著的额外存储开销。
- **局部性**：查询操作的局部性为 $O(1)$ ，意味着查询过程中的数据访问非常高效，避免了大量随机访问。
- **读取效率**：读取效率为 $P + \tilde{O}(\log \log N)$ ，这意味着读取操作主要受 PE-SSE 的页面效率影响，但也会有少量的对数级别额外开销。

3. 总结

- **定理 3** 证明了 ClipOSSE 中溢出项的数量可以被有效地控制，随着数据库大小的增长，溢出项的增长速度相对较慢，且上界为 $O(N / \log^d N)$ 。
- **定理 4** 证明了 Generic Local Transform（通过 ClipOSSE 和 PE-SSE）能够实现高效且安全的 SSE 方案，具有良好的存储效率、局部性和读取效率。

通过这些定理，我们得出结论：**通用本地变换** 提供了一个高效、隐私保护的数据存储与查询解决方案，适用于需要隐私保护且性能要求较高的场景。